Certified Compilation of Graph Coloring to Neutral-Atom Hardware

An Exact Gadgetized Reduction, a Heuristic Layout Optimizer, and a Type-and-Effect Discipline for the Pipeline

Research Draft

June 4, 2026

Abstract

Neutral-atom quantum processors solve a single native problem: maximum-weight independent set (MWIS) on a unit-disk graph (UDG), realized physically by the Rydberg blockade. They do *not* color graphs. We present a compilation pipeline that takes an arbitrary finite graph-coloring instance and produces a hardware-executable UDG–MWIS instance whose ground state decodes back to a proper coloring of the original graph, together with a small typed language that makes the pipeline's correctness static. The pipeline has two layers with a clean separation of concerns. The *primary path* is an exact, polynomial-time two-stage reduction: a textbook reduction of k-colorability to maximum independent set on an auxiliary graph G' of size nk, followed by a gadgetized embedding of G' into a weighted UDG using copy and crossing gadgets in the style of Nguyen et al. Correctness here is a once-proved meta-theorem, not a per-instance gamble: there is no chromatic-number inflation and the decoder is an exact inverse. The genuine engineering uncertainty therefore moves out of correctness and into *geometry*: atom count, array area, crossing number, and analog-solve quality. The *secondary path* is a suite of seven polynomial-time geometric heuristics; we prove an impossibility result that disqualifies them as a stand-alone coloring mechanism for general graphs and repurpose them as the *layout optimizer* that places and compacts the gadgetized instance under hardware constraints. We then introduce a Haskell-flavored surface language and a *type-and-effect discipline* in which every compilation stage is typed as an answer-preserving morphism carrying an executable decoder, and same-vertex transforms carry a *safety effect* bounding chromatic inflation; a single composition rule yields end-to-end soundness. The type discipline situates the work in the line of type-and-effect systems for behavioral policies developed by Gian-Luigi Ferrari and collaborators, to whom this paper is dedicated.

Keywords. graph coloring, unit-disk graphs, maximum independent set, Rydberg blockade, neutral-atom computing, gadget reductions, certified compilation, type-and-effect systems, QuEra, Bloqade.

Contents

1 Introduction

Graph coloring is a canonical NP-complete problem [20, 14] with broad practical reach, from register allocation and scheduling to frequency assignment. A recent and physically distinctive way to attack such combinatorial problems is the neutral-atom quantum platform, in which individually trapped atoms are excited to Rydberg states and interact through the *Rydberg blockade* [29, 17, 30]. The decisive feature of this platform is geometric: two atoms within a blockade radius r cannot both be excited, so the set of simultaneously excited atoms is precisely an *independent set* of the unit-disk graph (UDG) induced by atom positions. The device's analog ground state therefore encodes a *maximum-weight independent set* (MWIS) of that UDG [26, 11].

This single fact frames the entire compilation problem and is the pivot of this paper:

> *The hardware solves MWIS on a unit-disk graph. It does not color. Any pipeline that compiles coloring to this hardware must* reduce *coloring to UDG–MWIS, not merely* embed *the input graph geometrically.*

1.1 Two tempting designs, and why one is correct

There are two natural strategies for getting an arbitrary coloring instance onto this hardware.

Design A — geometric supergraph embedding. Place the original vertices in $\mathbb{R}^2$ and choose a radius r so that the induced UDG H is a *supergraph* of the input G, i.e. $E(G) \subseteq E(H)$. Then any proper coloring of H restricts to a proper coloring of G. This is attractive because the supergraph relation is trivially achievable and gives a one-line decoder. Section 4.6 develops the seven heuristics that make H as good as possible. However, Design A has two structural defects. First, we prove (Theorem 4.12) that a same-vertex UDG supergraph *cannot* bound chromatic-number inflation in general: the star $K_{1,n}$ has $\chi = 2$ but forces $\chi(H) \geq \lceil n/5 \rceil \to \infty$ in any UDG supergraph. Second, and independently, even a perfect H is not executable, because the device cannot color H; coloring a UDG on an MWIS machine requires iterated independent-set extraction, which is itself heuristic. Design A is thus heuristic twice and unbounded once.

Design B — exact gadgetized reduction. Reduce k-colorability of G to a *single* MWIS instance, then embed that instance into a weighted UDG the hardware can run. This is the standard reduction route, and it composes two exact, polynomial-time steps (Section 4): the classical colorability→independent-set reduction, and the Rydberg gadgetization of an arbitrary graph into a UDG via copy/crossing gadgets [25]. Here correctness is a once-proved meta-theorem with no inflation and an exact inverse decoder. The remaining uncertainty is not about *whether* the answer is correct but about *whether it fits the device and whether the analog dynamics finds the ground state* — both geometric/physical questions.

We adopt Design B as the primary path and retain Design A's heuristics in a strictly subordinate, and now sound, role: the *layout optimizer* that realizes the gadgetized instance under hardware spacing while minimizing atom count, area, and crossings. Because the gadget structure is fixed and exact, the heuristics can no longer affect correctness — only feasibility and solve quality. This relocation is the central design decision of the paper.

1.2 From a pipeline to a typed language

Once correctness lives entirely in a fixed chain of exact reductions, it becomes natural to make that chain *statically checkable*. The second half of the paper introduces a small surface language — Haskell-flavored, but deliberately restricted — whose types are not ordinary data types but *problem-types* ("a k-coloring instance on n vertices," "a weighted-IS instance with offset C,"

"a unit-disk instance that fits the device"), and whose arrows are *certified reductions* carrying executable decoders. The type system makes provenance, stage ordering, the device budget, and chromatic-safety part of typing, so that a well-typed program is a pipeline whose measured bitstring is guaranteed to decode to a correct coloring. The chromatic-safety annotation is a *type-and-effect* [22] in the lineage developed by Gian-Luigi Ferrari and collaborators [1, 10], to whose work this paper is dedicated.

1.3 Contributions

(1) A clean statement of the compilation problem that takes the hardware's native primitive (UDG–MWIS) as the target, and a two-layer pipeline that separates exact correctness from heuristic geometry (Sections 3–4).

(2) The exact primary reduction: colorability→MIS with a self-contained correctness proof and size nk (Theorem 4.2), composed with Rydberg UDG gadgetization (Theorem 4.4) to give an end-to-end exactness and decoding meta-theorem (Theorem 4.5), together with $O((nk)^2)$ atom-count and feasibility bounds (Section 4.4).

(3) An impossibility result (Theorem 4.12) that rigorously explains why the geometric-supergraph approach cannot be the correctness mechanism, justifying its demotion; and seven polynomial-time geometric heuristics, corrected and re-targeted as a hardware-aware *layout optimizer* with termination guarantees (Section 4.6).

(4) An integrated pipeline with worked examples, tables, figures, and a full complexity accounting (Sections 5–6).

(5) A Haskell-flavored surface language (Section 7) and a type-and-effect discipline (Section 8) that turns the reduction theorems into typing rules, with a pipeline-soundness theorem and a safety-effect monoid that answers the open question of whether transform order affects safety.

2 Literature Review

Graph coloring and its complexity. Deciding k-colorability for $k \geq 3$ is NP-complete [20, 14], and the chromatic number is hard even to approximate. Practical colorings are produced by greedy orderings such as largest-first [31] and by the saturation-degree heuristic DSATUR [5]; the clique number gives the basic lower bound $\omega(G) \leq \chi(G)$ [32]. We use DSATUR only as an auxiliary *witness* generator in the secondary path, never on the exact path.

Unit-disk graphs. UDGs are the intersection graphs of unit disks in the plane [9]. Recognition — deciding whether a given abstract graph is a UDG — is NP-hard [6, 19], which is precisely why we never attempt exact realization of the input graph. UDGs enjoy strong geometric locality: every clique lies inside a disk of radius r, and the packing of points in such a disk is bounded [18], a fact we use both for the impossibility result and for clique control. The exactly-realizable one-dimensional families are the unit/proper-interval graphs [28, 15].

Rydberg MWIS and gadgetized reductions. The identification of the Rydberg blockade ground state with MWIS on a UDG is due to Pichler et al. [26] and was demonstrated at scale on hundreds of atoms by Ebadi et al. [11]. Because native UDGs are geometrically restricted, Nguyen et al. [25] introduced copy and crossing gadgets that encode MWIS on an *arbitrary* graph as MWIS on a weighted UDG laid out on a grid, with a known value offset and a structural decoder. Our primary path is exactly the composition of the classical colorability→MIS reduction

with this gadgetization; the hardware target is QuEra's neutral-atom machine [33, 16], programmed through the Bloqade simulation stack. Variational and analog quantum optimization more broadly is surveyed in [7, 27].

Spectral and force-directed layout. Our layout optimizer draws on classical embedding machinery: the graph Laplacian and its eigenvectors [8, 12], Laplacian eigenmaps [2], force-directed drawing [13], multidimensional scaling [21], and gradient-based optimization [24, 4], with Pareto selection over the radius [23]. These are standard tools; our contribution is not the tools but their re-targeting to a *correctness-neutral* role behind an exact reduction.

Type-and-effect disciplines and certified transformation. Our type system (Section 8) is a *type-and-effect* system in the lineage initiated by Lucassen and Gifford [22] and developed extensively by Gian-Luigi Ferrari and his collaborators, who used types and effects to make security and resource-usage policies static and statically enforceable [1, 10, 3]. In that tradition an *effect* soundly over-approximates a computation's runtime behavior so that a policy can be checked *before* execution; the slogan that base types certify well-formedness while effects certify a behavioral policy is directly inherited from this line of work. We adopt exactly this idea for a different invariant — *chromatic safety* — annotating each same-vertex transform with a safety effect drawn from a monoid (safe / $\mathsf{asafe}(c)$) that bounds chromatic inflation, so that the order-sensitivity of safety becomes a checkable property rather than a conjecture (Section 8.6). It is a pleasure to dedicate this connection, and the paper, to Gian-Luigi Ferrari, in whose honor this special issue appears: the certified-reduction calculus developed here is, in spirit, a behavioral type-and-effect system, transplanted from the world of secure service composition to the world of quantum-hardware compilation.

3 Problem Formulation

We consider finite simple undirected graphs $G = (V, E)$ with $n = |V|$, $m = |E|$.

Definition 3.1 (Proper k-coloring; chromatic number)**.** A proper k-coloring is a map $c : V \to \{1, \dots, k\}$ with $c(u) \neq c(v)$ for every $\{u, v\} \in E$. The chromatic number $\chi(G)$ is the least k admitting one.

Definition 3.2 (Unit-disk graph)**.** Given a placement $p : V \to \mathbb{R}^2$ and radius $r > 0$, the induced UDG $\mathrm{UDG}(p, r)$ has an edge $\{u, v\}$ iff $\|p(u) - p(v)\|_2 \leq r$. A weighted UDG additionally assigns a weight $w : V \to \mathbb{R}_{>0}$.

Definition 3.3 (Independent set; MWIS)**.** An independent set of H is a vertex set with no internal edge. For weights w, the maximum-weight independent set value is $\mathrm{MWIS}(H, w) = \max\{\sum_{v \in S} w(v) : S \text{ independent}\}$. For unit weights this is the independence number $\alpha(H)$.

Hardware primitive. A neutral-atom array with atoms at positions $p(v)$ and blockade radius r realizes $\mathrm{UDG}(p, r)$; under appropriate driving its many-body ground state encodes $\mathrm{MWIS}(\mathrm{UDG}(p, r), w)$, where w is set by local detunings [26, 11]. The device constraints are a minimum atom spacing $d_{\min}$, a maximum blockade radius $r_{\max}$, a finite array area $A_{\max}$, and a maximum atom count $N_{\max}$ [33].

The compilation problem. Given (G, k) and the device parameters, produce (i) a weighted UDG instance A realizable on the array, and (ii) a decoder δ, such that from any MWIS of A measured by the device, δ recovers a proper k-coloring of G whenever one exists, and otherwise soundly reports non-k-colorability. The compilation itself must be polynomial time; only the analog solve is left to the physics.

The two-layer pipeline. Figure 1 shows the design. The primary (exact) path reduces coloring to an *abstract* weighted UDG–MWIS instance and decodes the measured independent set back to a coloring. The secondary (heuristic) path is the geometric engine that turns the abstract gadget instance into concrete atom coordinates obeying the hardware constraints, and is also available as a stand-alone — but only chromatic-non-decreasing — coloring backend for comparison.

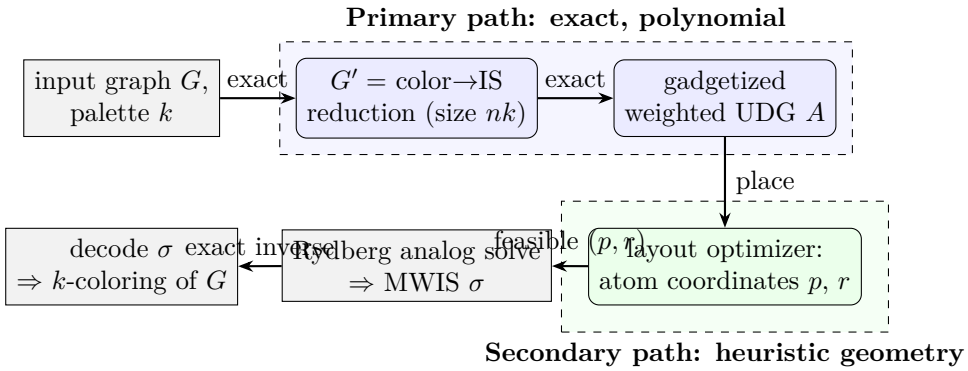

Figure 1: The two-layer pipeline. Correctness lives entirely in the blue (exact) path; the green (heuristic) path determines only hardware feasibility and analog-solve quality.

4 The Transformation, Step by Step

The primary path is the composition of two exact reductions:

$$\underbrace{G, k}_{\text{coloring}} \xrightarrow{\text{Stage 1}} \underbrace{G'}_{\text{MIS instance, size } nk} \xrightarrow{\text{Stage 2}} \underbrace{A = (p, r, w)}_{\text{weighted UDG}} \xrightarrow{\text{solve}} \sigma \xrightarrow{\text{decode}} \text{coloring of } G.$$

4.1 Stage 1: Colorability → Independent Set

Definition 4.1 (Color-choice graph)**.** Given $G = (V, E)$ and palette size k, the *color-choice graph* $G' = (V', E')$ has vertex set $V' = V \times \{1, \ldots, k\}$, and edge set $E' = E'_{\text{choice}} \cup E'_{\text{conflict}}$ where

$$E'_{\text{choice}} = \{\{(v, i), (v, j)\} : v \in V,\ 1 \le i < j \le k\}, \qquad \text{(a } k\text{-clique per vertex)}$$
$$E'_{\text{conflict}} = \{\{(u, i), (v, i)\} : \{u, v\} \in E,\ 1 \le i \le k\}. \qquad \text{(same-color conflicts)}$$

All vertices carry unit weight. Then $|V'| = nk$ and $|E'| = n\binom{k}{2} + mk$.

The choice-clique enforces "at most one color per vertex"; the conflict edges enforce "adjacent vertices differ in color."

Theorem 4.2 (Exactness of Stage 1)**.** *For every graph G and every $k \ge 1$,*

$$\alpha(G') = n \quad \Longleftrightarrow \quad G \text{ is } k\text{-colorable},$$

and $\alpha(G') \le n$ always. Moreover, independent sets of G' of size n are in explicit bijection with proper k-colorings of G.

Proof. For each v, the k vertices $\{(v, i)\}_i$ form a clique (E'_{choice}), so any independent set S contains at most one slot per original vertex; hence $|S| \le n$.

($\Leftarrow$) Let c be a proper k-coloring. Put $S = \{(v, c(v)) : v \in V\}$, so $|S| = n$. S is independent: it has exactly one slot per vertex (no choice edge is internal), and for any $\{u, v\} \in E$ we have $c(u) \neq c(v)$, so no conflict edge $\{(u, c(u)), (v, c(v))\}$ exists — conflict edges only join *equal* color indices. Thus $\alpha(G') \geq n$, and with $\alpha(G') \leq n$ we get $\alpha(G') = n$.

($\Rightarrow$) Let S be independent with $|S| = n$. By the choice-clique bound S has exactly one slot per vertex, defining $c(v) = i$ where $(v, i) \in S$. For $\{u, v\} \in E$, if $c(u) = c(v) = i$ then $\{(u, i), (v, i)\} \in E'_{\text{conflict}}$ would be internal to S, contradicting independence. Hence c is proper.

The two maps $c \mapsto S$ and $S \mapsto c$ are mutually inverse. □

Remark 4.3 (Fixed-k vs. chromatic number). A single value k tests k-colorability. The chromatic number is obtained by a sweep: the least k with $\alpha(G') = n$ equals $\chi(G)$. Binary search over $k \in \{1, \ldots, n\}$ costs $O(\log n)$ solves.

4.2 Stage 2: Independent Set → Unit-Disk MWIS

G' is an abstract graph and is generally not a UDG. Stage 2 embeds it into a weighted UDG that the hardware can realize, using the copy/crossing gadget construction of Nguyen et al. [25], itself building on the Rydberg MWIS encoding of Pichler et al. [26]. We use it as a black box with a precise interface.

Theorem 4.4 (Gadgetized UD-MWIS embedding [25])**.** *There is a polynomial-time procedure* gadgetize *that maps any vertex-weighted graph $G' = (V', E', w')$ to a weighted unit-disk graph $A = (p, r, w)$ on a grid, together with a designated set of* logical *atoms $\ell : V' \hookrightarrow V(A)$ and a constant $C \geq 0$, such that*

$$\mathrm{MWIS}(A, w) = \mathrm{MWIS}(G', w') + C,$$

and every maximum-weight independent set S_A of A restricts on the logical atoms to a maximum-weight independent set $\sigma = \ell^{-1}(S_A \cap \ell(V'))$ of G'. The number of atoms is $|V(A)| = O(|V'|^2)$ in the worst case, governed by the number of edge crossings in the chosen planar layout of G'.

The construction places the $|V'|$ logical atoms along a diagonal and routes each edge of G' as a chain of *copy* gadgets that propagate an atom's occupation; wherever two routed edges must cross, a *crossing* gadget passes both signals through without spurious coupling. Weights are chosen so that the ground-state independent set selects logical atoms exactly according to a maximum independent set of G'. We treat the exact gadget weights as fixed by [25]; choosing or re-deriving a custom gadget set is an engineering fork (Section 10).

4.3 Decoding and End-to-End Correctness

Decoding composes the two inverses. Given a hardware-measured independent set S_A of A: (1) restrict to logical atoms and apply ℓ^{-1} to obtain $\sigma \subseteq V'$ (Stage-2 inverse); (2) if $|\sigma| = n$, read the coloring $c(v) = i$ for the unique $(v, i) \in \sigma$ (Stage-1 inverse). The full chain is the following meta-theorem, proved once for all instances.

Theorem 4.5 (End-to-end exactness)**.** *Let $A = \mathsf{gadgetize}(G')$ for G' the color-choice graph of (G, k). Then:*

(i) $\mathrm{MWIS}(A, w) = n + C$ *if and only if G is k-colorable.*

(ii) Any maximum-weight independent set of A with logical-atom weight n decodes, via ℓ^{-1} followed by the Stage-1 readout, to a proper k-coloring of G.

(iii) The construction and the decoder both run in time polynomial in n and k, and the decoder is an exact inverse of the construction — there is no chromatic-number inflation.

Proof. By Theorem 4.4, $\mathrm{MWIS}(A, w) = \mathrm{MWIS}(G', w') + C = \alpha(G') + C$ (unit weights). By Theorem 4.2, $\alpha(G') = n$ iff G is k-colorable, and $\alpha(G') \leq n$ always; this gives (i). For (ii), a maximum independent set of A restricts (Theorem 4.4) to a maximum independent set σ of G'; when its size is n, the Stage-1 bijection (Theorem 4.2) reads σ off as a proper coloring. Polynomiality in (iii) is immediate: Stage 1 is $O(nk + mk)$ to build, Stage 2 is polynomial by Theorem 4.4, and both readouts are linear in their outputs. Exactness of each inverse was established in the respective theorems, and exactness composes. □

Remark 4.6 (Where uncertainty actually lives). Theorem 4.5 is unconditional: it never assumes the analog solve succeeds. The two real risks are (a) *feasibility* — whether A fits inside $N_{\max}$ atoms and $A_{\max}$ area (Section 4.4), and (b) *analog-solve quality* — whether the Rydberg dynamics reaches the MWIS ground state rather than a low-lying excited state. Neither is a correctness question about the reduction, and a returned independent set is always *verifiable*: $|\sigma| = n$ certifies a valid coloring, $|\sigma| < n$ certifies only that the solver did not find a witness.

4.4 Size and Feasibility Bounds

Proposition 4.7 (Atom budget). *For an instance (G, k) with $|V| = n$, the gadgetized UDG A uses $|V(A)| = O((nk)^2)$ atoms in the worst case, and at least nk (the logical atoms). The instance is hardware-feasible only if $|V(A)| \leq N_{\max}$ and the layout fits within $A_{\max}$ under spacing $d_{\min}$.*

Proof. Stage 1 produces $|V'| = nk$ logical vertices; Stage 2 contributes $O(|V'|^2)$ atoms by Theorem 4.4. The lower bound is the logical-atom count. □

The quadratic blow-up against a few-hundred-atom device [33] is the practical bottleneck, and it is exactly what the layout optimizer of Section 4.6 attacks. Two preprocessing knobs reduce the effective size before gadgetization:

- **Graph simplification.** Removing vertices of degree $< k$ (they are always colorable last), contracting degree-2 chains, and decomposing on cut vertices and biconnected components all shrink n without changing k-colorability.
- **Crossing minimization.** Because Stage 2's atom count is dominated by edge crossings, a good planar-ish layout of G' (or of G before expansion) directly lowers $|V(A)|$. This is a geometric objective — the province of the heuristics.

4.5 Worked Example: 3-Coloring a Small Graph

Example 4.8. Let G be the 4-cycle C_4 with vertices $v_1v_2v_3v_4$ and $k = 3$. Stage 1 builds G' on $V' = \{(v_a, i) : a \in \{1, 2, 3, 4\},\ i \in \{1, 2, 3\}\}$, $|V'| = 12$. Each vertex contributes a choice-triangle on its three slots; each edge of C_4 contributes three conflict edges (one per color). A proper 3-coloring such as $c = (1, 2, 1, 2)$ corresponds to the independent set $\{(v_1, 1), (v_2, 2), (v_3, 1), (v_4, 2)\}$ of size $4 = n$, and conversely. Since $\chi(C_4) = 2 \leq 3$, Stage 1 reports $\alpha(G') = 4$, so G is 3-colorable. Gadgetizing G' yields a weighted UDG with 12 logical atoms plus routing/crossing atoms; measuring its MWIS and applying ℓ^{-1} recovers the size-4 set, whose Stage-1 readout is the coloring $(1, 2, 1, 2)$. No inflation occurs at any step.

4.6 Secondary Path: Heuristic Geometry and the Layout Optimizer

We now develop the geometric heuristics. We first prove why they cannot serve as the coloring-correctness mechanism, then state the role they actually play.

4.6.1 Why geometric supergraphs cannot bound inflation

Definition 4.9 (Safe supergraph; color inflation)**.** A UDG $H = \mathrm{UDG}(p, r)$ on the same vertex set V as G is a *safe supergraph* if $E(G) \subseteq E(H)$. The *false edges* are $E_{\text{false}} = E(H) \setminus E(G)$, and the *color-inflation ratio* is $\rho = \widehat{\chi}(H)/\widehat{\chi}(G)$, where $\widehat{\chi}$ is a chromatic estimate (e.g. DSATUR).

Proposition 4.10 (Safety ⇒ decoder correctness)**.** *If $E(G) \subseteq E(H)$ and κ is a proper coloring of H, then κ restricted to V is a proper coloring of G. Consequently $\chi(G) \le \chi(H)$.*

Proof. Every edge of G is an edge of H, so the endpoints receive distinct colors under κ. The inequality follows since a proper coloring of H is one of G. □

So safe supergraphs never *lower* χ. The problem is the other direction.

Lemma 4.11 (Neighborhood packing in a UDG)**.** *In any $\mathrm{UDG}(p, r)$ and for any vertex v, the open neighborhood $N(v)$ has independence number $\alpha(H[N(v)]) \le 5$.*

Proof. All neighbors of v lie in the closed disk of radius r about $p(v)$. An independent set among them consists of points pairwise at distance $> r$. Partition the disk into six $60°$ sectors about $p(v)$; two points in the same sector at distance $\le r$ from the center are within $\le r$ of each other (the largest pairwise distance inside a $60°$ circular sector of radius r is r), hence adjacent. So at most one independent point lies per sector, giving at most 6; a standard refinement excluding the center configuration sharpens the bound to 5 [18]. □

Theorem 4.12 (Supergraph inflation is unbounded)**.** *There is a family of graphs with $\chi = 2$ on which every safe UDG supergraph has $\chi(H) \to \infty$. Concretely, for the star $K_{1,n}$,*

$$\chi(K_{1,n}) = 2, \qquad \chi(H) \;\ge\; \left\lceil \tfrac{n}{5} \right\rceil \quad \textit{for every safe UDG supergraph } H.$$

Proof. $K_{1,n}$ is bipartite, so $\chi = 2$. Let H be a safe UDG supergraph with center c and leaves L, $|L| = n$. Safety forces every edge $\{c, \ell\}$ into H, so all leaves lie within distance r of $p(c)$, i.e. $L \subseteq N(c)$. In any proper coloring of H, each color class is an independent set of H, hence an independent set within $N(c)$, which by Lemma 4.11 has size ≤ 5. Covering the n leaves therefore needs at least $\lceil n/5 \rceil$ color classes, so $\chi(H) \ge \lceil n/5 \rceil$. □

Remark 4.13. Theorem 4.12 is the rigorous reason the supergraph route cannot be "usually safe" for general graphs, and hence cannot be the primary path. It also pinpoints a sufficient structural escape: if the per-vertex neighborhood independence $\sigma(G) = \max_v \alpha(G[N(v)])$ is ≤ 5, the obstruction vanishes, which is the natural precondition under which supergraph safety claims can be scoped (and which reappears as a typing side-condition in Section 8.6).

4.6.2 The re-targeted role: layout optimizer for the gadget instance

Given Theorem 4.12, the heuristics are *not* used to make $\chi(H) \approx \chi(G)$. Instead, the gadgetized instance A of Section 4.2 fixes the required adjacency *exactly*; the heuristics solve the residual *geometric realization* problem:

> *place the atoms of A in $\mathbb{R}^2$ so that the realized UDG equals the prescribed gadget adjacency, while obeying $d_{\min}$, $r \le r_{\max}$, area $\le A_{\max}$, and minimizing atom count / crossings / chain length.*

Crucially, in this role **the heuristics cannot affect correctness**: the gadget structure and weights are fixed by Theorem 4.4, so a poor layout can only fail feasibility (does not fit) or degrade analog-solve quality (harder ground state), never the decoded answer. This is the soundness payoff of Design B.

The optimizer's loss is therefore an *adjacency-realization* loss, not a G-mimicking loss. For the target adjacency E_A of A:

$$L_{\text{layout}}(p,r) = \underbrace{\sum_{\{u,v\}\in E_A} \max(0, d_{uv} - r)^2}_{\text{prescribed edges within } r} + \lambda \sum_{\{u,v\}\notin E_A} \max(0, r+\epsilon-d_{uv})^2 + \mu \sum_{u\neq v} \max(0, d_{\min}-d_{uv})^2, \tag{1}$$

where $d_{uv} = \|p(u) - p(v)\|_2$ and the third term enforces the hardware spacing floor. The seven heuristics below initialize, drive, repair, and tune this optimization.

4.6.3 The seven heuristics

H1 — Geometric loss minimization. Optimize (1) jointly over coordinates p and radius r by gradient descent [24, 4], $p \leftarrow p - \eta\nabla_p L$, $r \leftarrow r - \eta\nabla_r L$. After convergence apply a **hard feasibility check**: every prescribed edge realized, every non-edge excluded, all pairwise distances $\geq d_{\min}$. If a prescribed edge is missing, set $r \leftarrow \max_{\{u,v\}\in E_A} d_{uv} + \delta$ and rebuild. Cost $O(|V(A)|^2)$ per step.

H2 — Crossing/chain-aware weighting. Edges of A that route through many crossing gadgets are the expensive ones. Weight the loss terms by gadget-chain length so the optimizer prioritizes compacting long routed edges, the analogue of the color-danger weighting of the supergraph setting (DSATUR [5] is used there to score danger; here the score is geometric chain cost). This directly attacks the $O((nk)^2)$ bottleneck of Proposition 4.7.

H3 — Clique/over-packing control. Since $\omega(H) \leq \chi(H)$ and since every UDG clique is geometrically local (contained in some $N[v]$), spurious dense clusters both waste atoms and can corrupt the realized adjacency. Detect them within local neighborhoods in $O(|V(A)|\Delta^2)$ and push offenders apart. The hardware itself bounds clique size:

Proposition 4.14 (Hardware-bounded cliques)**.** *In a physical neutral-atom UDG with blockade radius r and minimum spacing $d_{\min}$, the maximum clique size obeys $\omega_{\max} \leq \lfloor \pi r^2/(c\, d_{\min}^2) \rfloor$, where c is the planar packing constant ($c \approx 0.9$) [18].*

This gives an a priori cap on local density that depends only on physical parameters [17, 30] and is a useful invariant for the layout.

H4 — Laplacian eigenmap initialization. Initialize p_0 from the Fiedler vector and the next Laplacian eigenvector [12, 8, 2], $p_0(v_i) = (\phi_2(i), \phi_3(i))$. This places adjacent atoms near and well-separated parts far, giving gradient descent a good basin [13]. It is an initializer for H1, not a stand-alone step; cost $O(|V(A)|^2)$ dense, or $O(|V(A)| \cdot t)$ for t eigenvectors via sparse solvers.

H5 — Local repair with a monotone potential. After global optimization, fix residual violations locally by pushing/pulling offending pairs along their connecting line. To guarantee termination, gate every accepted move by strict decrease of the integer-valued potential

$$\Phi = \sum_{\{u,v\}\in\mathcal{V}} w_{uv},$$

where $\mathcal{V}$ is the current set of adjacency violations and $w_{uv} \geq 1$ are integer weights. Since $\Phi \geq 0$ is integer and drops by ≥ 1 per accepted step, at most $\Phi_0 \leq |V(A)|^2 \max w$ steps occur — a genuine polynomial bound (unlike a vague "until convergence").

H6 — Radius sweep and Pareto selection. With p fixed, sort all pairwise distances and sweep r. The *minimum feasible radius* $r^\star = \max_{\{u,v\}\in E_A} d_{uv}$ is the smallest r realizing all prescribed edges; for $r < r^\star$ the layout is infeasible. Over $r \in [r^\star, r_{\max}]$ select a Pareto point minimizing (realized non-edges, atom area, r) subject to $r \le r_{\max}$ [23]. Cost $O(|V(A)|^2 \log |V(A)|)$ to sort plus $O(|V(A)|^2)$ per evaluated radius.

H7 — Inflation/quality as evaluation, not objective. Discrete quality measures (number of realized non-edges, decoded solution gap) are piecewise-constant in (p, r) and have no useful gradient, so they must be used to *evaluate* and *select* layouts (e.g. at the H6 sweep) rather than as differentiable objectives. For the optional stand-alone supergraph backend, the same caveat applies to ρ.

Table 1 summarizes the seven, their role in the optimization, and their per-pass cost.

	Heuristic	Role in the layout optimization	Cost (per pass)
H1	Geometric loss minimization	drive Eq. (1) by gradient descent	$O(N^2)$
H2	Crossing/chain-aware weighting	prioritize long routed edges	$O(N^2)$
H3	Clique / over-packing control	break spurious dense clusters	$O(N\Delta^2)$
H4	Laplacian eigenmap init	seed p_0 in a good basin	$O(N^2)$ / $O(Nt)$
H5	Local repair (Φ-gated)	fix residual violations, terminating	$\le \Phi_0 = O(N^2 \max w)$
H6	Radius sweep + Pareto	pick $(p, r^\star)$, $r^\star \le r_{\max}$	$O(N^2 \log N)$
H7	Quality as evaluation	select, do not differentiate	$O(N^2)$

Table 1: The seven heuristics as components of the layout optimizer, with $N = |V(A)|$ and Δ the maximum degree. None affects correctness (Section 4.6.2).

4.6.4 Corrected exactly-realizable families

For the optional stand-alone supergraph backend it is useful to know families that embed with *zero* false edges ($\rho = 1$). The naive folklore claims require correction.

Proposition 4.15 (Caterpillars, not all trees)**.** *Not every tree is a UDG: the star $K_{1,6}$ is a tree that is not a unit-disk graph (Theorem 4.12 with $n = 6$ forces false structure, and indeed $K_{1,6}$ has no UDG realization). However, every* caterpillar *(a tree whose non-leaf vertices form a path) is a proper-interval graph and hence realizes as a 1D UDG with $\rho = 1$.*

Proposition 4.16 (Unit-interval, not all interval graphs)**.** *The 1D unit-disk graphs are exactly the* unit-interval *(equivalently proper-interval) graphs [28, 15]. Not every interval graph qualifies: the claw $K_{1,3}$ is an interval graph but not unit-interval. The claim "interval graphs are 1D UDGs" must be restricted to unit-interval graphs.*

Remark 4.17. The earlier blanket assertions "all trees achieve $\rho = 1$," "all interval graphs are 1D UDGs," and an unconditional "bipartite UDGs achieve $\rho = 1$" are false or vacuous and are replaced here by Propositions 4.15–4.16; the bipartite claim is dropped as it presupposes the embedding it purports to construct.

5 Integrated Pipeline

Algorithm 1 states the end-to-end compiler. Steps 1–2 are the exact primary path; steps 3–8 are the heuristic layout of the resulting gadget instance; steps 9–10 solve and decode. Figure 2 shows the data flow.

Input: graph $G = (V, E)$, palette size k (or sweep for χ), hardware parameters $(d_{\min}, r_{\max}, A_{\max}, N_{\max})$.
Output: proper k-coloring of G, or a feasibility/solve-failure certificate.

1. **Stage 1 (exact):** build color-choice graph G'. *// $O(nk + mk)$; Thm. 4.2*
2. **Stage 2 (exact):** $A, \ell, C \leftarrow \mathsf{gadgetize}(G')$. *// poly; Thm. 4.4; check $|V(A)| \le N_{\max}$*
3. **H4:** $p_0 \leftarrow \text{LaplacianEigenmap}(A)$. *// initialize layout*
4. **H1–H2:** minimize $L_{\text{layout}}(p, r)$ for a fixed $T = \text{poly}(|V(A)|)$ budget. *// Eq. (1)*
5. **Feasibility check:** all prescribed edges realized, spacing $\ge d_{\min}$; if not, raise r or rebuild.
6. **H3:** break over-packed clusters. *// Φ-gated*
7. **H5:** local repair of residual violations. *// Φ-gated, terminates*
8. **H6:** radius sweep; pick Pareto $(p, r^\star)$ with $r^\star \le r_{\max}$, area $\le A_{\max}$.
9. **Solve:** run Rydberg MWIS on A at $(p, r^\star)$; measure S_A.
10. **Decode:** $\sigma \leftarrow \ell^{-1}(S_A)$; if $|\sigma| = n$ return the Stage-1 coloring readout, else report solve failure. *// Thm. 4.5*

Algorithm 1: Certified coloring-to-neutral-atom compiler

6 Complexity Analysis

Let $N = |V(A)| = O((nk)^2)$ be the atom count. Table 2 summarizes the per-stage cost. Every stage is polynomial; the exact path is low-degree polynomial, and the dominant cost is the layout optimization over N atoms.

Stage	Cost	Type	Notes
Stage 1 (color→IS)	$O(nk + mk)$	exact	build G'
Stage 2 (gadgetize)	poly(nk), $N = O((nk)^2)$	exact	Thm. 4.4
H4 Laplacian init	$O(N^2)$ (one-time)	heuristic	sparse: $O(Nt)$
H1/H2 optimization	$O(T\,N^2)$	heuristic	$T = \text{poly}(N)$ budget
H3 clique control	$O(N\Delta^2)$ per pass	heuristic	local
H5 local repair	$\le \Phi_0$ steps, $\Phi_0 = O(N^2 \max w)$	heuristic	Φ-gated, terminates
H6 radius sweep	$O(N^2 \log N)$	heuristic	$+\ O(N^2)$/radius
Decode	$O(N)$	exact	ℓ^{-1} + readout

Table 2: Per-stage complexity. The NP-hardness of UDG *recognition* [6] is never incurred: we never decide whether G is a UDG, only realize a prescribed gadget adjacency.

Proposition 6.1 (Polynomial compilation)**.** *With a fixed iteration budget $T = \text{poly}(N)$, Algorithm 1 runs in time polynomial in n and k for everything except the analog solve, and emits a hardware instance of $O((nk)^2)$ atoms together with an exact decoder.*

Proof. Sum the rows of Table 2; each is polynomial in $N = O((nk)^2)$ and $T = \text{poly}(N)$. Exactness of the emitted decoder is Theorem 4.5. □

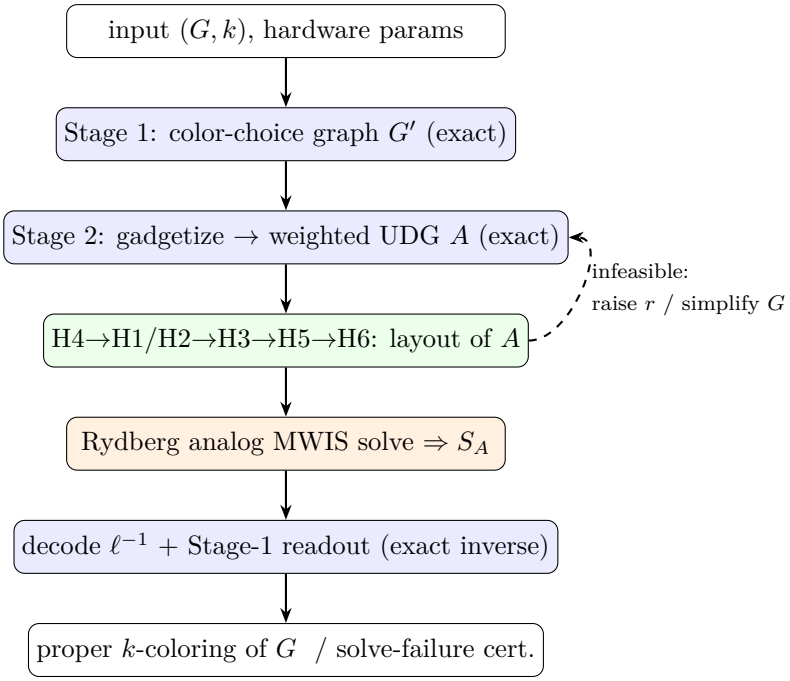

Figure 2: Data flow of Algorithm 1. Blue = exact, green = heuristic layout, orange = analog hardware. Only feasibility, never correctness, flows back.

7 A Language for Certified Coloring Compilation

Having fixed the semantics, we now give the surface language a programmer writes. It is *Haskell-flavored* — pure, statically typed, with algebraic data types, top-level type signatures written with `::`, layout-sensitive record syntax, and pipeline composition — but it is *not* a general-purpose language. The differences are deliberate and serve certification.

7.1 What we keep from Haskell, and what we change

7.2 Concrete grammar

The surface syntax is given below in EBNF. A program is a sequence of typed top-level bindings; the domain forms (`graph`, `palette`, `restrictions`, `transform`, `instance`, `compute`) are ordinary bindings whose right-hand side is a domain expression.

```
Program     ::= { Binding }

Binding     ::= [ Signature ] Definition
Signature   ::= Ident "::" Type
Definition  ::= Ident "=" Expr
              | "graph"        Ident "=" GraphExpr
              | "palette"      Ident "=" PaletteExpr
              | "restrictions" Ident "=" "{" { Restriction ";" } "}"
              | "transform"    Ident "=" TransformExpr
              | "instance"     Ident "=" InstanceExpr
              | "compute"      Ident "=" QueryExpr

-- Types: ordinary arrows, problem-types, and the certified-reduction arrow ~>
Type        ::= AType { "->" AType }                  -- ordinary functions
              | Problem "~>" Problem [ "!" Effect ]   -- certified reduction (+ effect)
AType       ::= Problem | "Graph" | "Palette" | "Result" | "(" Type ")"
Problem     ::= "Coloring" Nat Nat                    -- Col<n,k>
```

Haskell-like (kept)	Domain-specific (changed)
ADTs, records, `type` synonyms	The base "values" are *problems* (graphs, palettes, instances), not arbitrary data.
Top-level signatures `name :: T`	Types include *problem-types* $\mathsf{Col}, \mathsf{IS}, \mathsf{WIS}, \mathsf{UDG}, \mathsf{Dev}$ (Section 8) indexed by size, offset, radius, spacing.
Purity, referential transparency	Total: **no general recursion**; every program denotes a finite pipeline that provably terminates.
Function arrow `->`	A second arrow `~>`, the *certified-reduction* arrow, is a first-class type former carrying a decoder and an *effect*.
Type classes / constraints	A *guarantee lattice* and a *safety effect* $\{\mathsf{safe}, \mathsf{asafe}(c)\}$ replace ad-hoc constraints; subtyping lets a stronger certificate stand in for a weaker one.
»>-style composition	Composition threads decoders in reverse and sums effects; ill-ordered pipelines simply do not type.

Table 3: Design summary: a small total DSL whose type system is the static shadow of the reduction theorems.

```
              | "IS" Nat                                  -- IS<N>
              | "WIS" Nat Nat                            -- WIS<N,C>
              | "UDG" Nat Real Real                      -- UDG<N,r,d>
              | "Dev" Nat                                -- device-feasible
Effect      ::= "safe" | "asafe" Nat

GraphExpr   ::= "graph" "{" "vertices" "=" "[" VertexList "]" ","
                            "edges"    "=" "[" EdgeList "]" "}"
              | "udg" "{" "radius" "=" Real "," "edges" "=" "[" EdgeList "]" "}"
              | "apply" TransformRef "to" GraphRef

PaletteExpr ::= "colors" "[" ColorList "]" | "size" Nat

TransformExpr ::= "toUDG" "(" TransformMode { "," TransformOption } ")"
              | "encodeColoring" Nat                     -- exact gadget encoding
              | "superUDG"                               -- same-vertex supergraph
              | "colorChoice" Nat | "gadgetize" | "layout" Ident | "emit"
              | TransformExpr ">>>" TransformExpr        -- pipeline composition
              | "compose" "[" TransformRefList "]"

TransformMode   ::= "exact" | "preserve" Property
TransformOption ::= "radius" "=" Real | "keep" "=" "[" VertexList "]"
                  | "drop" "=" "[" VertexList "]" | "trackOrigin" "=" Bool
Property        ::= "kColorable" Nat | "chromatic" | "bipartite"

InstanceExpr ::= "color" GraphRef [ "using" PaletteRef ]
                 [ "subjectTo" RestrRef ] [ "via" TransformRef ]
QueryExpr    ::= "chromaticNumber" "(" GraphRef ")"

Restriction  ::= "precolor" Vertex ":" Color
              | "allow" Vertex ":" "[" ColorList "]"
              | "forbid" Vertex ":" "[" ColorList "]"
              | "sameColor" "(" Vertex "," Vertex ")"
              | "differentColor" "(" Vertex "," Vertex ")"
              | "useAtMost" Nat | "useExactly" Nat | "distinctOn" "[" VertexList "]"
```

```
Edge       ::= "(" Vertex "," Vertex ")"
Vertex     ::= Ident | Nat        Color ::= Ident | String
Ident      ::= letter { letter | digit | "_" }
Nat        ::= digit { digit }    Real ::= Nat [ "." Nat ]   Bool ::= "true"|"false"
```

7.3 Abstract syntax (Haskell)

The AST is itself written in Haskell, emphasizing the family resemblance. Note the `Ty` type carries a certified-reduction arrow indexed by source and target *problem-types* together with an `Effect`.

```
type Name = String ; type Vertex = String ; type Color = String
type Edge = (Vertex, Vertex) ; type Radius = Double

data Program = Program [Binding]
data Binding = Binding (Maybe Sig) Definition
data Sig     = Sig Name Ty

data Definition
  = DLet          Name Expr
  | DGraph        Name GraphExpr
  | DPalette      Name PaletteExpr
  | DRestrictions Name [Restriction]
  | DTransform    Name TransformExpr
  | DInstance     Name InstanceExpr
  | DQuery        Name QueryExpr

-- Problem-types (the kinds the certified-reduction arrow connects)
data Problem
  = Coloring Int Int          -- Col<n,k>
  | IS Int                    -- IS<N>
  | WIS Int Int               -- WIS<N,C>
  | UDG Int Double Double     -- UDG<N,r,d>
  | Dev Int                   -- device-feasible

data Ty
  = TFun Ty Ty                       -- ordinary  a -> b
  | TReduce Problem Problem Effect   -- certified reduction  A ~> B ! e
  | TGraph | TPalette | TResult

data Effect = Safe | ASafe Int

data TransformExpr
  = ToUDG TransformMode [TransformOption]
  | EncodeColoring Int        -- exact gadget encoding
  | SuperUDG                  -- same-vertex supergraph (carries a safety effect)
  | ColorChoice Int | Gadgetize | Layout Name | Emit
  | Pipe TransformExpr TransformExpr      -- f >>> g
  | Compose [Name]

data TransformMode = Exact | Preserve Property
data Property      = KColorable Int | Chromatic | Bipartite
data TransformOption
  = RadiusOpt Double | KeepVertices [Vertex] | DropVertices [Vertex]
  | TrackOrigin Bool
```

```
data GraphExpr   = GraphLit [Vertex] [Edge] | UdgLit Radius [Edge]
                 | ApplyTransform Name Name
data PaletteExpr = ExplicitPalette [Color] | PaletteSize Int
data Restriction = Precolor Vertex Color | Allow Vertex [Color]
                 | Forbid Vertex [Color] | SameColor Vertex Vertex
                 | DifferentColor Vertex Vertex | UseAtMost Int
                 | UseExactly Int | DistinctOn [Vertex]
data InstanceExpr = InstanceExpr
  { instGraph :: Name, instPalette :: Maybe Name
  , instRestrictions :: Maybe Name, instTransform :: Maybe Name }
data QueryExpr = ChromaticNumber Name
```

7.4 Example program

The following program compiles a 3-coloring of a small graph through the exact gadget pipeline, and separately exercises the (typed, weaker) supergraph path. The type signatures are checked by the discipline of Section 8.

```
-- input graph and palette ------------------------------------------------
gInput :: Graph
gInput = graph { vertices = [1,2,3,4,5]
               , edges    = [(1,2),(2,3),(3,1),(1,4),(3,5)] }

pNamed :: Palette
pNamed = colors [red, blue, green]

restrictions rMain = { precolor 1 : red;
                       differentColor (1,2);
                       useAtMost 3 }

-- the exact primary pipeline, fully typed as a chain of certified reductions
encode3 :: Coloring 5 3 ~> Dev 225
encode3 = colorChoice 3 >>> gadgetize >>> layout opts >>> emit

-- run it: the measured bitstring decodes to a proper coloring (soundness)
solve3 :: Result
solve3 = color gInput using pNamed subjectTo rMain via encode3

-- secondary (supergraph) path: SAME vertices, carries a safety effect.
-- Only well-typed when sigma(G) <= 5 ; here it elaborates to `! asafe 1`.
super :: Coloring 5 3 ~> Coloring 5 4 ! asafe 1
super = superUDG

-- a pure query
chiInput :: Int
chiInput = chromaticNumber (gInput)
```

8 The Type System

This section gives the type discipline that makes the pipeline's correctness static. It targets *one thing*: the faithfulness of the compilation *stages*. Ordinary programming errors — scope, arity, base types, totality of plain functions — are delegated to a standard Hindley–Milner core with refinements; we write $\Gamma \vdash e : T$ for that judgment and never re-derive it. What we add is a small *certified-reduction calculus* on top.

Key point. The slogan, inherited from type-and-effect systems [22, 1]: *base types say a program is well-formed; problem-types and effects say a* ***reduction*** *is faithful.* We build only the second.

8.1 Problem-types (the sorts)

A *problem-type* names a class of decision/optimization instances together with the indices we must track to state preservation.

$A, B \;::=$	$\mathsf{Col}\langle n, k\rangle$	k-coloring instance, $n = \lvert V\rvert$
	$\mathsf{Col}\langle n, k \mid \beta\colon B\rangle$	boundary-precolored (β proper on $B \subseteq V$)
	$\mathsf{IS}\langle N\rangle\{\, P\,\}$	independent-set instance, N nodes, refinement P
	$\mathsf{WIS}\langle N,\, C\rangle$	weighted IS, N nodes, additive offset C
	$\mathsf{UDG}\langle N,\, r,\, d\rangle$	*exactly* realized unit-disk inst. (radius r, spacing d)
	$\widetilde{\mathsf{UDG}}\langle N,\, r,\, d \mid f\rangle$	*approximately* realized, f residual false edges
	$\mathsf{Dev}\langle N\rangle \quad (N \le N_{\max})$	device-feasible instance

Each A comes with a *solution domain* $\mathrm{Sol}(A)$ and an *optimum* $\mathrm{opt}(A) \in \mathbb{R} \cup \{\bot\}$. The refinement P on IS is where Stage-1 exactness lives; the canonical one is

$$P_{\mathrm{s1}} \;\equiv\; (\alpha = n \iff G \text{ is } k\text{-colorable}) \;\wedge\; \alpha \le n.$$

Remark 8.1 (Why $\widetilde{\mathsf{UDG}}$ exists). UDG is the placement in which the realized unit-disk graph equals the *prescribed* gadget adjacency *exactly.* When the layout optimizer leaves false edges, the object is honestly a *different* graph, so it gets a different type, $\widetilde{\mathsf{UDG}}$, and — crucially — $\widetilde{\mathsf{UDG}}$ **is not a legal source for the exact decoder** (Section 8.3). The type is what stops us from silently reading an answer off a corrupted layout.

8.2 The certified-reduction judgment

Definition 8.2 (Decoder; total on optima)**.** A *decoder* from B to A is a partial map $\delta : \mathrm{Sol}(B) \rightharpoonup \mathrm{Sol}(A)$. It is *total on optima*, written $\mathsf{tot\text{-}opt}(\delta)$, if δ is defined on every $s \in \mathrm{Sol}(B)$ with value $\mathrm{opt}(B)$ and maps it to a value-$\mathrm{opt}(A)$ solution of A.

Definition 8.3 (Certified reduction)**.** The judgment $\vdash\ \tau\ :\ A \overset{\delta}{\rightsquigarrow} B$ asserts that τ takes an instance of A to an instance of B, that δ is a decoder $B \rightharpoonup A$ with $\mathsf{tot\text{-}opt}(\delta)$, and that $\mathrm{opt}(A)$ is recoverable as $\delta(s^\star)$ for any optimum $s^\star$ of B. Equivalently: *solve B on the device, apply δ, get the answer to A.* This is the surface arrow `A ~> B` of Section 7.

8.3 Per-stage typing rules

The premises above each line are the proof obligations; in the implementation they are discharged either by a once-and-for-all proof or by a brute-force oracle on small instances (Section 8.7).

Stage 1 — color-choice.

$$\frac{n = |V(G)| \qquad k \ge 1}{\vdash\ \mathsf{colorChoice}_k\ :\ \mathsf{Col}\langle n, k\rangle \overset{\mathsf{decodeColoring}}{\rightsquigarrow} \mathsf{IS}\langle nk\rangle\{P_{\mathrm{s1}}\}} \quad \text{S1}$$

$\mathsf{decodeColoring}$ is total on size-n independent sets and partial elsewhere — exactly $\mathsf{tot\text{-}opt}$.

Stage 2 — gadget.

$$\frac{C = 2\,|E(G')|}{\vdash\ \mathsf{gadgetize}\ :\ \mathsf{IS}\langle N\rangle\{P\} \overset{\mathsf{decodeLogical}}{\rightsquigarrow} \mathsf{WIS}\langle N + 2|E(G')|,\ C\rangle}\ \text{S2}$$

The offset C is carried in the type so "subtract C" is type-directed; decodeLogical (keep logical atoms) is tot-opt.

Stage 3 — layout (branches on realizability). Write $\mathsf{realizes}(p, r)$ for "the unit-disk graph of (p, r) has edge set exactly E_A" and $\mathsf{false}(p, r)$ for the residual false edges.

$$\frac{\mathsf{realizes}(p,r) \qquad \mathsf{spacing}(p) \geq d}{\vdash\ \mathsf{layout}(p,r)\ :\ \mathsf{WIS}\langle N, C\rangle \overset{\mathrm{id}}{\rightsquigarrow} \mathsf{UDG}\langle N, r, d\rangle}\ \textsc{S3-exact}$$

$$\frac{\mathsf{miss}(p,r) = \varnothing \qquad f = |\mathsf{false}(p,r)| > 0}{\vdash\ \mathsf{layout}(p,r)\ :\ \mathsf{WIS}\langle N, C\rangle \rightsquigarrow \widetilde{\mathsf{UDG}}\langle N, r, d \mid f\rangle}\ \textsc{S3-approx}$$

S3-exact carries the identity decoder (geometry never relabels a solution) and lands in UDG; S3-approx carries *no* certified decoder and lands in $\widetilde{\mathsf{UDG}}$. The only way to turn $\widetilde{\mathsf{UDG}}$ back into a legal UDG is the metric repair transform that re-routes false edges through copy/crossing gadgets,

$$\frac{}{\vdash\ \mathsf{nguyen}\ :\ \widetilde{\mathsf{UDG}}\langle N, r, d \mid f\rangle \overset{\mathsf{decodeRoute}}{\rightsquigarrow} \mathsf{WIS}\langle N', C'\rangle}\ \textsc{Repair}$$

after which S3-exact can be attempted again.

Emit — device budget.

$$\frac{N \leq N_{\max}}{\vdash\ \mathsf{emit}\ :\ \mathsf{UDG}\langle N, r, d\rangle \overset{\mathrm{id}}{\rightsquigarrow} \mathsf{Dev}\langle N\rangle}\ \textsc{Fit}$$

Key point. The pipeline can reach Dev **only through** UDG, never through $\widetilde{\mathsf{UDG}}$. So "the machine measured a bitstring" is connected to "a correct coloring" by a *chain of* tot-opt *decoders* — or it is not connected at all, and the type checker says so.

8.4 Composition and the soundness theorem

$$\frac{\vdash\ \tau_1 : A \overset{\delta_1}{\rightsquigarrow} B \qquad \vdash\ \tau_2 : B \overset{\delta_2}{\rightsquigarrow} C}{\vdash\ \tau_2 \circ \tau_1\ :\ A \overset{\delta_1 \circ \delta_2}{\rightsquigarrow} C}\ \textsc{Comp}$$

Theorem 8.4 (Pipeline soundness)**.** *If* $\vdash\ \Pi\ :\ \mathsf{Col}\langle n, k\rangle \overset{\delta}{\rightsquigarrow} \mathsf{Dev}\langle N\rangle$ *is derivable (so* $N \leq N_{\max}$ *and every layer is realized* exactly*), then for any device measurement* m *with value* $\mathrm{opt}(\mathsf{Dev}\langle N\rangle)$*, the decoded* $\delta(m)$ *is a proper* k*-coloring of* G *if one exists, and otherwise the pipeline reports "not* k*-colorable" soundly.*

Proof sketch. By induction on the derivation using Comp: each arrow is answer-preserving with a tot-opt decoder, and the composite of tot-opt decoders is tot-opt, so $\delta = \delta_{\mathrm{s1}} \circ \delta_{\mathrm{s2}} \circ \mathrm{id} \circ \mathrm{id}$ maps an optimum bitstring back through WIS (strip offset C), IS (read one slot per vertex), to a coloring; the refinement P_{s1} supplies the iff. The only non-identity geometric step is admitted by S3-exact, whose premise $\mathsf{realizes}(p, r)$ guarantees the realized graph *is* A, so its optima coincide. □

Corollary 8.5 (Ill-ordered pipelines are ill-typed)**.** *Matching the error catalogue (pipeline-stage ordering):*

- gadgetize *before* colorChoice*: S2 expects* IS*, is given* Col *— no rule applies.*
- emit *before* layout*: Fit expects* UDG*, is given* WIS*.*
- *reading an answer off a false-edged layout: there is no* tot-opt *decoder out of* $\widetilde{\mathsf{UDG}}$ *(only Repair, which goes* back *to* WIS*).*

8.5 Refinement A: certificate grade

We expose the witness-relative vs. true-χ distinction as a *grade* $g \in \{\mathsf{w}, \chi\}$ on coloring conclusions, with $\mathsf{w} \sqsubseteq \chi$: $\mathsf{Col}\langle n, k\rangle^{\mathsf{w}}$ is *witness-relative* (a carried coloring κ certifies $\chi(G) \le |\kappa|$ in poly time — the default), while $\mathsf{Col}\langle n, k\rangle^{\chi}$ is *exact* ($\chi(G) = k$ as a discharged proof obligation). All stage rules are grade-polymorphic and preserve the grade; only an explicit prove_{χ} step (a discharged lower-bound proof, e.g. a clique) lifts $\mathsf{w} \Rightarrow \chi$. Theorem 8.4 holds at grade w already: *soundness of compilation does not require knowing the true chromatic number.*

8.6 Refinement B: the safety effect (supergraph path)

The secondary (supergraph) path rewrites G on the *same* vertices by adding edges, decoded by restriction. Such a transform may inflate χ, so it carries a *safety effect* s — a genuine type-and-effect in the sense of [1]:

$$\vdash\ t\ :\ \mathsf{Col}\langle n, k\rangle \overset{\mathsf{restrict}}{\underset{s}{\rightsquigarrow}} \mathsf{Col}\langle n, k'\rangle, \qquad s \in \{\,\mathsf{safe},\ \mathsf{asafe}(c)\,\}.$$

safe means χ is preserved; asafe(c) means t never lowers χ and $\chi(\text{target}) \le \chi(\text{source}) + c$.

Effect monoid.

$$\mathsf{safe} \oplus \mathsf{safe} = \mathsf{safe}, \qquad \mathsf{safe} \oplus \mathsf{asafe}(c) = \mathsf{asafe}(c), \qquad \mathsf{asafe}(c) \oplus \mathsf{asafe}(c') = \mathsf{asafe}(c + c').$$

Composition combines decoders *and* effects:

$$\frac{\vdash t_1 : A \overset{\delta_1}{\underset{s_1}{\rightsquigarrow}} B \qquad \vdash t_2 : B \overset{\delta_2}{\underset{s_2}{\rightsquigarrow}} C}{\vdash t_2 \circ t_1 : A \overset{\delta_1 \circ \delta_2}{\underset{s_1 \oplus s_2}{\rightsquigarrow}} C} \quad \textsc{Comp-S}$$

Mandatory precondition (the impossibility theorem as a side-condition). A safe/asafe(c) introduction is *only* derivable under the neighborhood bound $\sigma(G) = \max_v \alpha(G[N(v)]) \le 5$; otherwise the only available type is unsafe (no χ certificate). This is exactly the $\chi(K_{1,n}) = 2$ vs. $\chi(H) \ge \lceil n/5 \rceil$ obstruction of Theorem 4.12, refusing to type.

Remark 8.6 (Does transform order affect safety?). Because $\oplus$ sums the bumps c but the *carrier graph* differs between orders, two orderings of the same multiset of asafe$(\cdot)$ transforms are type-equal only if their accumulated bounds *and* side-conditions both match. The type therefore *records* order-sensitivity as a difference in the derived effect — turning the project's open question into a checkable property rather than a conjecture. Correctness, by contrast, composes order-independently because each arrow is answer-preserving (Theorem 8.4); only *cost* and the safety bound are order-sensitive.

8.7 The discipline is the static shadow of the oracle

Every rule corresponds to a function in (or planned for) the Haskell validation harness, and every premise to a property a brute-force oracle checks; the type system promotes those runtime checks to static obligations. The recommended minimal core to mechanize first is S1, S2, Comp, and Theorem 8.4: the smallest closed fragment that already makes "the measured bitstring decodes to a correct coloring" a *typed* guarantee.

9 Discussion and Limitations

What is and isn't guaranteed. The reduction is exact and verifiable: a decoded set of size n is a certified coloring. What is *not* guaranteed is that the analog device reaches the MWIS ground state, or that a large instance fits the array. These are physics and capacity questions, cleanly separated from correctness by Theorem 4.5 and from typing by Theorem 8.4.

The size wall. The $O((nk)^2)$ atom budget is the binding constraint against present devices of a few hundred atoms [33]; in the worst case this admits only a handful of original vertices. We treat this as an honest limitation, not the headline: the value of the framework — the exact reduction, the impossibility result, the type-and-effect discipline — is independent of $N_{\max}$ and durable as hardware scales. The leverage against the wall is preprocessing (graph simplification, crossing minimization, decomposition) and the layout optimizer; quantifying the achievable reduction on structured instances is the main empirical question this design opens.

Status of the secondary path. The stand-alone supergraph backend is retained only for theory and comparison. By Theorem 4.12 it is unbounded on general graphs, and it additionally requires iterated independent-set extraction to color on hardware. Its practically valuable contribution is its geometric machinery, which the primary path reuses as the layout optimizer.

10 Future Work

Two engineering forks remain open: *which gadget set* to adopt (the verbatim construction of [25] versus a custom set, affecting what must be re-proved), and confirming the *weighted* MWIS interface end-to-end. On the language side, the next steps are to mechanize the minimal core (Section 8.7) in a proof assistant or as liquid refinements, to add the decomposition layer (precolor / split / stitch with a typed device budget $(wk)^2 \leq N_{\max}$), and to wire the type checker to the validation harness so that absent proofs fall back to the oracle on small instances. Empirically, we plan an evaluation in Bloqade against QuEra-class parameters [33], measuring atom count, area, crossing number, and ground-state success probability across structured and random instances.

11 Conclusion

Neutral-atom hardware solves UDG–MWIS, not coloring, and that single fact selects the architecture. Compiling coloring exactly means *reducing* it to UDG–MWIS via the classical colorability→independent-set reduction composed with Rydberg copy/crossing gadgetization — a once-proved, inflation-free, polynomial pipeline with an exact decoder. The geometric heuristics that a supergraph approach would have leaned on for correctness are provably unable to bound chromatic inflation on general graphs; their proper and sound home is the layout optimizer that fits the exact gadget instance onto the device. On top of this semantic substrate sits a small Haskell-flavored language whose type-and-effect discipline makes provenance, stage ordering, the device budget, and chromatic safety static, so that a well-typed program is a certified compiler. This separation — exact correctness in the reduction, heuristic effort in the geometry, behavioral guarantees in the types — is the foundation we offer, and the connection to type-and-effect systems is one we are glad to dedicate to Gian-Luigi Ferrari.

References

[1] Massimo Bartoletti, Pierpaolo Degano, Gian-Luigi Ferrari, and Roberto Zunino. Local policies for resource usage analysis. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 31(6):1–43, 2009.

[2] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps and spectral techniques for embedding and clustering. *Advances in Neural Information Processing Systems*, 2003.

[3] Chiara Bodei, Pierpaolo Degano, Gian-Luigi Ferrari, and Corrado Priami. Comparing flow algebras for process calculi. In *Concurrency Theory (CONCUR)*. Springer, 1998.

[4] Stephen Boyd and Lieven Vandenberghe. Convex optimization. *Cambridge University Press*, 2004.

[5] Daniel Brelaz. New methods to color the vertices of a graph. *Communications of the ACM*, 22(4):251–256, 1979.

[6] H Breu and DG Kirkpatrick. On the complexity of recognizing unit disk graphs. *Journal of Computational Geometry and Theory of Applications*, 1998.

[7] Marco Cerezo et al. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, 2021.

[8] Fan RK Chung. *Spectral Graph Theory*, volume 92. American Mathematical Society, 1997.

[9] Brent N Clark, Charles J Colbourn, and David S Johnson. Unit disk graphs. *Discrete Mathematics*, 86(1-3):165–177, 1990.

[10] Pierpaolo Degano, Gian-Luigi Ferrari, and Letterio Galletta. A two-component language for adaptation: design, semantics, and program analysis. *IEEE Transactions on Software Engineering*, 42(6):505–522, 2016.

[11] Sepehr Ebadi, Alexander Keesling, Madelyn Cain, Tout T Wang, Harry Levine, Dolev Bluvstein, Giulia Semeghini, Ahmed Omran, J-G Liu, Rhine Samajdar, et al. Quantum optimization of maximum independent set using rydberg atom arrays. *Science*, 376(6598):1209–1215, 2022.

[12] Miroslav Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.

[13] Thomas MJ Fruchterman and EM Reingold. Graph drawing by force-directed placement. *Software: Practice and experience*, 21(11):1129–1164, 1991.

[14] Michael R Garey and David S Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[15] Martin Charles Golumbic. Algorithmic graph theory and perfect graphs. *Academic Press*, 1980.

[16] TM Graham et al. Multi-qubit entanglement and algorithms on a programmable neutral-atom quantum processor. *Nature*, 604:457–462, 2022.

[17] Luc Henriet et al. Quantum computing with neutral atoms. *Quantum*, 5:528, 2021.

[18] A Heppes. On the packing density of circles and spheres. *Mathematika*, 2003.

[19] Daniel Jungck et al. On the complexity of udg recognition and related problems. In *Computational Geometry*, 2021.

[20] Richard M Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Springer, 1972.

[21] Joseph B Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29(1):1–27, 1964.

[22] John M Lucassen and David K Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 47–57. ACM, 1988.

[23] Kaisa Miettinen. Nonlinear multiobjective optimization. 2014.

[24] Yurii Nesterov. Introductory lectures on convex optimization: A basic course. *Applied Optimization*, 87, 2004.

[25] Minh-Thi Nguyen, Jin-Guo Liu, Jonathan Wurtz, Mikhail D Lukin, Sheng-Tao Wang, and Hannes Pichler. Quantum optimization with arbitrary connectivity using rydberg atom arrays. *PRX Quantum*, 4(1):010316, 2023.

[26] Hannes Pichler, Sheng-Tao Wang, Leo Zhou, Soonwon Choi, and Mikhail D Lukin. Quantum optimization for maximum independent set using rydberg atom arrays. *arXiv preprint arXiv:1808.10816*, 2018.

[27] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.

[28] Fred S Roberts. Recent progresses in combinatorics. *Academic Press*, 1969.

[29] Mark Saffman, Timothy G Walker, and Klaus Molmer. Quantum computing with neutral atoms. *Reviews of Modern Physics*, 82(3):2313, 2010.

[30] Pascal Scholl et al. Quantum computing hardware for neutral atoms. *Nature*, 605:202–206, 2022.

[31] DJA Welsh and MB Powell. An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, 10(1):85–89, 1967.

[32] Douglas B West. *Introduction to Graph Theory*. Prentice Hall, 2nd edition, 2001.

[33] Jonathan Wurtz, Alexei Bylinskii, Boaz Braverman, Jesse Amato-Grill, Sergio H Cantu, et al. Aquila: QuEra's 256-qubit neutral-atom quantum computer. *arXiv preprint arXiv:2306.11727*, 2023.